RAG-Based Customer Support Assistant

(High level Design)

Name: Sakshi Suresh Dhurde

College: P R Pote College Of Engineering And Management, Amravati.

Course: Internship Project

Date: April 2026

Id: IN226096402

Problem Definition

Traditional customer support systems such as rule-based chatbots or FAQ bots are not capable of handling **real-world, dynamic user queries** effectively. They typically rely on:

- Hardcoded responses
- Keyword matching
- Predefined decision trees

Because of this, they fail in scenarios where:

- The user question is phrased differently from stored FAQs
- Information is buried inside long documents (PDFs, manuals, policies)
- Context is required to understand the correct answer

  Now, this system solves that by building a **Retrieval-Augmented Generation (RAG) based Customer Support Assistant** that uses a company knowledge base (PDF) to generate accurate, contextual responses.

◈ Scope of System

The system supports:

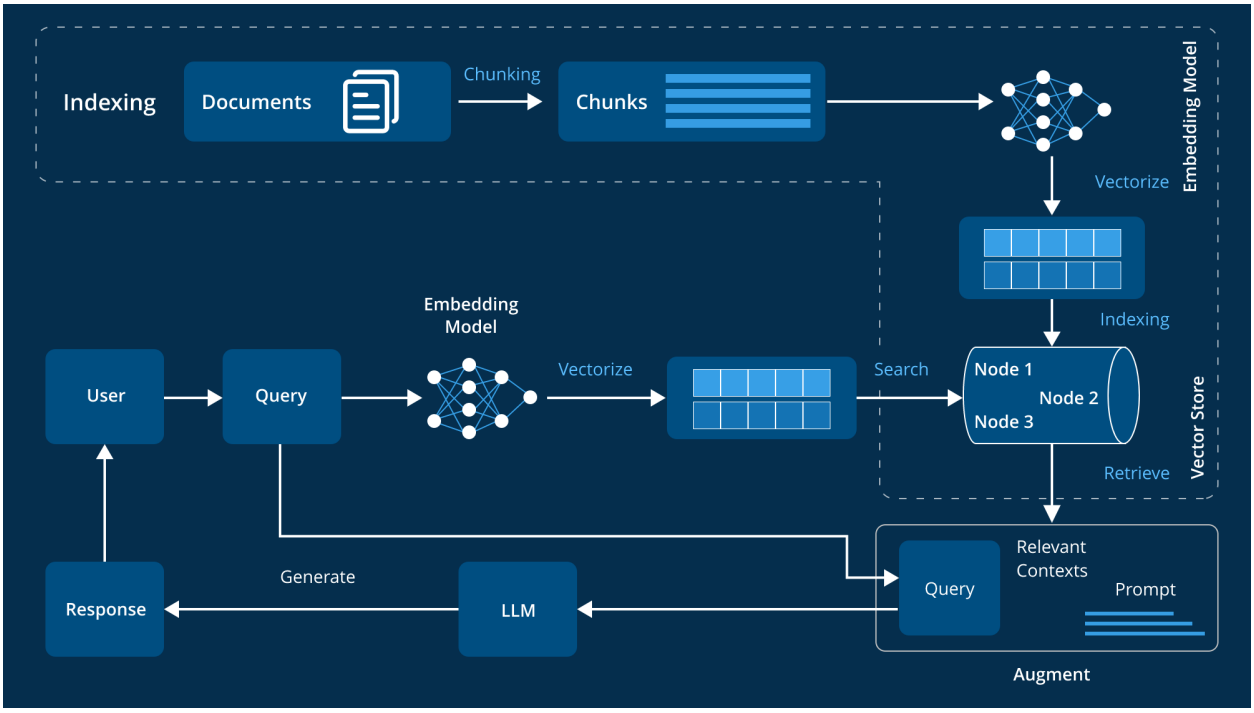
- PDF document ingestion

- Semantic search using embeddings
- Context-aware question answering
- Workflow-based decision making (LangGraph)
- Human-in-the-loop escalation for uncertain queries
- Web-based chat interface (Streamlit)

---

2. Architecture Diagram (Textual)

```
User (Streamlit UI)
      |
      ▼
Query Input Layer
      |
      ▼
LangGraph Orchestrator
      |
┌──────────────────┬──────────────────┐
▼                  ▼                  ▼
Intent             Retrieval          HITL Check
Classifier         (ChromaDB)         (Confidence)
      |
      ▼
Embedding Model
      |
      ▼
Vector Database (ChromaDB)
      |
      ▼
LLM Response Generator
      |
      ▼
Final Answer to User
```



3. Component Description

📄 Document Loader

- Loads PDF using PyPDF
- Extracts raw text

✂️ Chunking Strategy

- Paragraph-based chunking
- Removes noise and merges meaningful sections
- Avoids splitting policies incorrectly

🧠 Embedding Model

- SentenceTransformers (all-MiniLM-L6-v2)
- Converts text into vector representations

🗄️ Vector Store

- ChromaDB (persistent storage)

- Stores embeddings + metadata

🔎 **Retriever**

- Performs similarity search
- Returns top-k relevant chunks

🤖 **LLM**

- Generates natural language response
- Uses retrieved context

🔗 **Graph Workflow Engine**

- LangGraph-based orchestration
- Controls flow between nodes

🔀 **Routing Layer**

- Decides:
    - direct answer
    - retrieval needed
    - escalation to human

🧑‍💻 **HITL Module**

- Escalates low-confidence queries
- Sends query to human agent
- Returns human response to user

---

4. Data Flow

🔷 **PDF → Answer Flow**

PDF Upload → Text Extraction → Chunking → Embeddings → ChromaDB Storage

🔷 Query Lifecycle

User Query → Embedding → Vector Search → Context Retrieval → LLM → Answer → HITL Check → Response

---

5. Technology Choices

🗄 ChromaDB

- Lightweight vector database
- Fast similarity search
- Easy local deployment

🔗 LangGraph

- Manages multi-step workflows
- Supports conditional routing
- Enables HITL integration

🤖 LLM Choice

- OpenAI / Mistral / local LLM
- Used for natural language generation

⚙ Additional Tools

- Streamlit → UI
- SentenceTransformers → embeddings
- PyPDF → document parsing

---

6. Scalability Considerations

📚 Large Documents

- Chunking optimization required
- Hierarchical chunking for large PDFs

⚡ Query Load

- Caching frequent queries
- Batch embedding requests

⏱ **Latency**

- Precomputed embeddings
- Reduced retrieval top-k
- Async LLM calls